Université Abdelmalek Essaâdi

École nationale des sciences appliquées d'Al Hoceima

Département Mathématiques et Informatique

TDIA2

Compte Rendu

TP JEE – MVC2 (Hibernate)

Réalisé par :

Fatima Achbout

Professeur:

Mohamed Cherradi

1. Introduction :

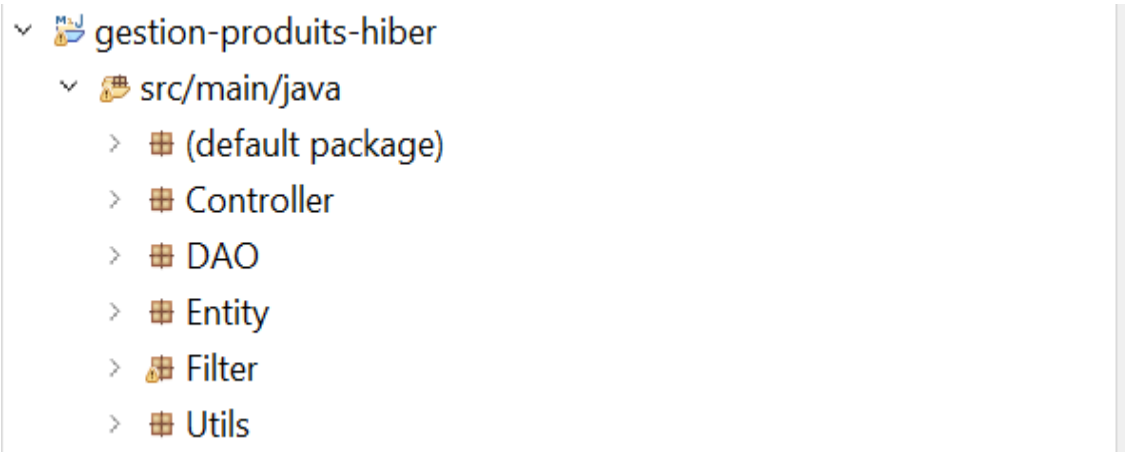
Ce travail pratique a pour objectif de nous familiariser avec le développement d'applications web en Java EE en utilisant les technologies Hibernate et les Servlets dans une architecture MVC2.

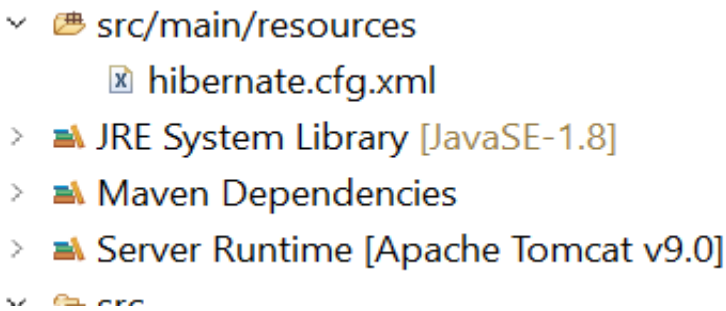
Dans ce projet, nous avons développé une mini application de gestion des produits permettant d'effectuer les opérations CRUD (Créer, Lire, Modifier, Supprimer) sur les produits. L'application intègre également un système d'authentification avec gestion des rôles (administrateur et utilisateur), afin de contrôler l'accès aux différentes fonctionnalités.

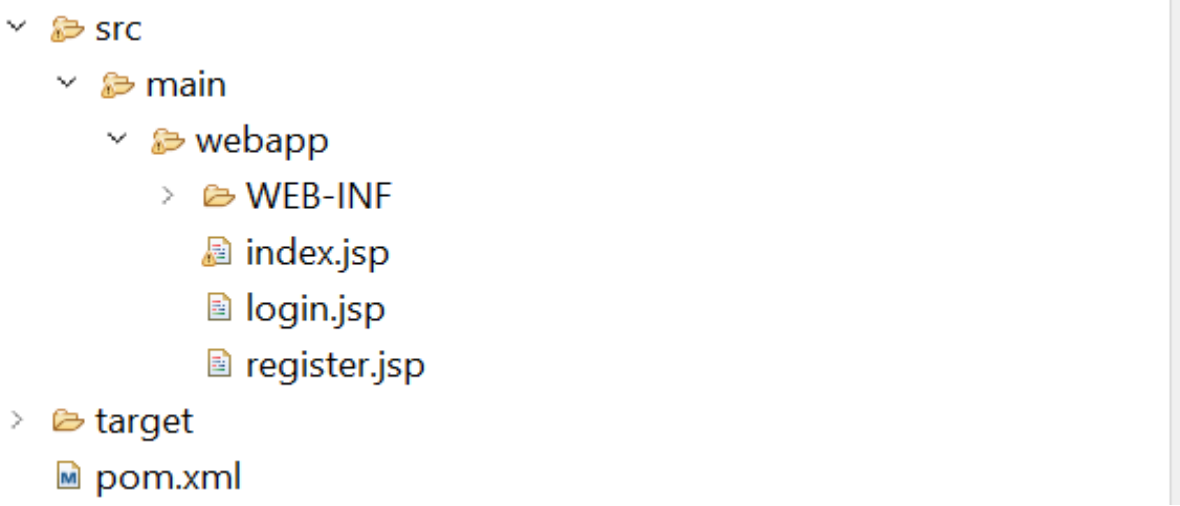
Le projet repose sur une architecture MVC2, où une seule Servlet centrale joue le rôle de contrôleur principal. Cette dernière gère les requêtes HTTP et les redirige vers les différentes couches de l'application (DAO, modèles et vues JSP).

2. Architecture de l'application :

La réalisation de cette application a été effectuée en plusieurs étapes afin de respecter une architecture claire basée sur le modèle MVC2 et d'intégrer les technologies Hibernate et Java Servlet.







3. Réalisation :

3.1. Etape 1 :

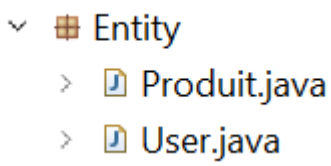
Dans un premier temps, j'ai commencé par la configuration du projet Maven en ajoutant les dépendances nécessaires pour Hibernate, MySQL Connector dans le fichier pom.xml. Cette étape a permis de préparer l'environnement de développement et d'assurer la communication avec la base de données.

```
<!-- ================= HIBERNATE ================= -->
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>5.6.15.Final</version>
</dependency>
```

```
<!-- ================= MYSQL ================= -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.33</version>
</dependency>
```

3.2. Etape 2 :

Création du package Entity qui contient les classes métiers principales du projet, à savoir User et Produit. Ces classes représentent les tables de la base de données et sont utilisées par Hibernate pour la gestion des données.



3.3. Etape 3 :

Configuration de Hibernate à travers le fichier hibernate.cfg.xml, dans lequel j'ai défini les paramètres de connexion à la base de données MySQL (URL, username, password, dialecte), ainsi que les entités à mapper (User et Produit). Ce fichier permet également la création automatique des tables via Hibernate.

```
<session-factory>

    <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
    <property name="connection.url">jdbc:mysql://localhost:3306/ ?createDatabaseIfNotExist=t
    <property name="dialect">org.hibernate.dialect.MySQL5Dialect</property>
    <property name="hbm2ddl.auto">update</property>
    <property name="show_sql">true</property>

    <property name="connection.username" /property>
    <property name="connection.password"> /property>
    <mapping class="Entity.Produit"/>
    <mapping class="Entity.User"/>

</session-factory>
```

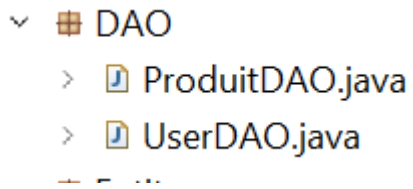
3.4. Etape 3 :

Création d'une classe utilitaire HibernateUtil dans le package Utils, dont le rôle est de centraliser la création du SessionFactory. Cette classe permet de gérer efficacement les connexions à la base de données en créant une seule instance réutilisable dans toute l'application.

```
public class HibernateUtil {

    private static SessionFactory sessionFactory;

    static {
        try {
            Configuration cfg = new Configuration();
            cfg.configure("hibernate.cfg.xml");

            sessionFactory = cfg.buildSessionFactory();

        } catch (Exception e) {
            e.printStackTrace();
            throw new ExceptionInInitializerError("Hibernate init failed");
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

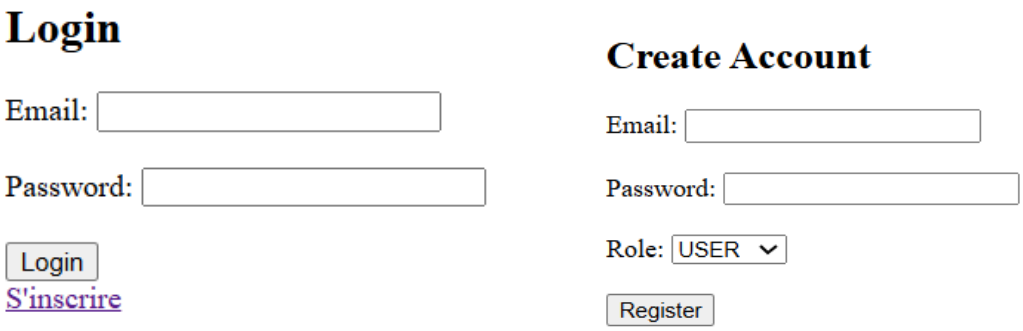
3.5. Etape 4 :

Création du package DAO contenant les classes UserDAO et ProduitDAO. Ces classes sont responsables de toutes les opérations liées à la base de données, notamment l'ajout, la suppression, la mise à jour et la récupération des données. Elles permettent de séparer la logique métier de la couche de persistance.



3.6. Etape 5 :

Création des vues JSP, notamment index.jsp, login.jsp et register.jsp. Ces pages permettent à l'utilisateur d'interagir avec le système. La page index est utilisée pour la gestion des produits, tandis que les pages login et register gèrent l'authentification.



Index.jsp si l'utilisateur n'est pas Admin :

Index.jsp si l'utilisateur est un admin :

Gestion des Produits MVC2

Bienvenue, nihad@gmail.com | Déconnexion

Nom: Description: Prix: Ajouter

ID: Rechercher

ID	Nom	Description	Prix	Actions
1	Iphone	mobile phone	40000.0	Modifier Supprimer
3	Sumsung	Marque des tele	80000.0	Modifier Supprimer

3.7. Etape 5 :

Pour sécuriser l'application, j'ai implémenté des filtres afin de vérifier si l'utilisateur est connecté avant d'accéder aux pages protégées. Si l'utilisateur n'est pas authentifié, il est redirigé vers la page de login.

```
if (path.contains("login.jsp") ||
    path.contains("register.jsp") ||
    "login".equals(action) ||
    "register".equals(action)) {
    System.out.println("Filter is running...");
    chain.doFilter(request, response);
    return;
}


if (session == null || session.getAttribute("user") == null) {
    res.sendRedirect("login.jsp");
    return;
}
System.out.println("Filter is running...");

chain.doFilter(request, response);
```

Enfin, la gestion des rôles (admin et user) a été réalisée à l'aide des sessions HTTP. Lors de la connexion, les informations de l'utilisateur sont stockées dans la session, ce qui permet de contrôler les permissions et d'afficher les fonctionnalités adaptées dans la page index.jsp.

4. Conclusion :

Ce travail pratique m'a permis de mettre en application les concepts fondamentaux du développement web en Java EE, notamment à travers l'utilisation des Servlets, de Hibernate et de l'architecture MVC2.

Grâce à ce projet, j'ai pu comprendre le rôle de chaque couche dans une application web structurée : la couche Entity pour représenter les données, la couche DAO pour gérer l'accès à la base de données, la couche Controller pour traiter les requêtes, et enfin la couche View pour l'interaction avec l'utilisateur via les pages JSP.

L'intégration de Hibernate a facilité la gestion de la persistance des données avec MySQL en simplifiant les opérations CRUD. De plus, la mise en place des sessions et des filtres m'a permis de mieux comprendre les mécanismes d'authentification et de sécurité dans une application web.

Enfin, ce TP a été une expérience enrichissante qui m'a aidé à renforcer mes compétences techniques en développement Java EE, ainsi qu'à mieux comprendre la séparation des responsabilités dans une architecture MVC2, ce qui est essentiel pour le développement d'applications professionnelles évolutives et maintenables.